Ders 12 Çalışma Özeti

Ders 12 Çalışma Özeti

Genel Konular

- Dersin biçimi: öğrenci sunumları
  - Chris Richardson'ın "Microservices Patterns" kitabından iki konu sunulur: mikroservislerin dağıtımı (deployment) ve Bölüm 11 "Üretime hazır servisler geliştirme". İkinci sunum önceden kaydedilmiş bir video olarak izletilir. Hoca ilk sunumdan sonra dağıtım stratejilerini kendi anlatımıyla özetler; ikinci sunumun ortasında ve sonunda kısa değerlendirmeler yapar.
- Üretim ortamının dört yeteneği (ekrandan, 00:06:57 ve 00:08:17)
  - Servis yönetim arayüzü (service management interface): geliştiricinin servis oluşturmasını, güncellemesini ve yapılandırmasını sağlar; ideal olarak bir komut satırı ya da grafik arayüzün kullandığı bir API ile.
  - Çalışma zamanı servis yönetimi (runtime service management): istenen sayıda servis örneğinin çalıştığından emin olur; çöken ya da isteklere cevap veremeyen servisi yeniden başlatır; makine çökerse servisi başka bir makinede başlatır.
  - İzleme (monitoring): geliştiriciye servislerin ne yaptığını gösterir (pano/dashboard ve uyarı/alerting); sorun varsa uyarır ve sebebini bildirir.
  - İstek yönlendirme (request routing): kullanıcıdan gelen istekleri servislere yönlendirir.
- Dağıtım kalıbı 1: Host başına birden çok servis örneği (Multiple Service Instances per Host)
  - En yaygın ve en eski yaklaşım: bir sanal ya da fiziksel makinede birden çok servis çalışır.
  - Artıları: servisler aynı sunucuyu ve işletim sistemini paylaştığı için kaynak kullanımı görece verimlidir; dağıtım hızlıdır (Java'da JAR ya da WAR dosyasını, Ruby ya da Node.js'te kaynak kodunu kopyalamak yeterlidir); ek yük olmadığı için servis hızlı başlar.
  - Eksileri (ekrandan, 00:10:00): servis örneği başına kaynak sınırlanamaz, bir sızıntı bütün belleği tüketip diğer servisleri aç bırakabilir; aynı süreçte çalışan servisler arasında yalıtım yoktur, hatalı bir servis diğerlerini de keser; dağıtımı yapan ekip servislerin en küçük ayrıntısını bilmek zorunda olduğu için hata riski yüksektir; servis örneklerinin makinelere yerleşimini belirlemek zordur.
- Dağıtım kalıbı 2: Servisi sanal makine olarak dağıtmak (Service as a VM) (ekrandan, 00:10:20)
  - Servis bir sanal makine imajı olarak paketlenir, her servis örneği ayrı bir VM olarak dağıtılır. Slayttaki şekil: servis kodu, dağıtım hattındaki VM imaj oluşturucu tarafından bir AMI'ye (VM imajı) dönüştürülür; çalışma zamanında bu imaj bir otomatik ölçekleme grubundaki EC2 örnekleri olarak çalışır; istekler Elastic Load Balancer üzerinden gelir.
  - Artıları: imaj servisin bütün bağımlılıklarını içerir, kurulum hataları ortadan kalkar; paketlenen servis bir kara kutu olur ve her yerde değişiklik gerekmeden dağıtılabilir; servisler birbirinden tam yalıtılır; her servise sabit CPU ve bellek ayrılarak birinin diğerinin kaynağını çalması engellenir; AWS gibi altyapılarla otomatik ölçekleme ve yük dengeleme yapılabilir.
  - Eksileri: kaynak kullanımı daha az verimlidir; kullanılsa da kullanılmasa da ücret ödendiği için maliyet artar; VM imajı büyük olduğu için oluşturulması, ağda taşınması ve başlatılması (işletim sisteminin açılması) daha uzun sürer; işletim sistemi sürüm

ve yama yönetimi geliştiriciye kalır.

- Dağıtım kalıbı 3: Servisi konteyner olarak dağıtmak (Service as a Container) (ekrandan, 00:13:05)
  - Konteyner, işletim sistemi seviyesinde bir sanallaştırma mekanizmasıdır: genellikle bir, bazen birkaç süreç, onu diğer konteynerlerden yalıtan bir korumalı alanda (sandbox) çalışır. Örneğin Java servisi çalıştıran bir konteyner tipik olarak JVM sürecinden oluşur.
  - Konteynerdeki süreç kendi makinesindeymiş gibi çalışır: kendi IP adresi olduğu için port çakışması olmaz (bütün Java süreçleri 8080'i dinleyebilir), kendi kök dosya sistemi vardır.
  - VM yaklaşımının bütün artılarını taşır; ek olarak daha hafif olduğu için derlenmesi, taşınması ve başlatılması daha hızlıdır; servis başına CPU ve bellek sınırı konabilir; tam yalıtım sağlar.
  - Eksileri: imajların yönetimi geliştiricidedir; hazır bir konteyner çözümü (ör. Amazon'unki) kullanılmıyorsa konteyner altyapısının yönetimi de geliştiriciye kalır.
- Dağıtım kalıbı 4: Sunucusuz dağıtım (Serverless deployment) (ekrandan, 00:14:30)
  - Java, Node.js ve Python servislerini destekler; AWS Lambda yaygın bir örnektir. Servis ZIP dosyası olarak paketlenip durumsuz (stateless) bir Lambda fonksiyonuna yüklenir. Lambda, istekleri karşılamaya yetecek kadar servis örneğini otomatik çalıştırır; ücret istek başına, harcanan süre ve belleğe göre ödenir.
  - Artıları: AWS hizmetleriyle bütünleşiktir; sistem yönetimi işini geliştiricinin üzerinden alır; ihtiyaç anında çalışır, kapasite tahmini gerekmez.
  - Eksileri: özellikle Java'da başlatma gecikmesi olabilir, anında tepki gereken servislere uygun olmayabilir; çalışma süresi sınırlıdır (sunumda 300 saniye).
- Hocanın dağıtım özeti: sanallaştırma seviyesi adım adım yükselir
  - Tek makinede bütün servisler, mimari dışında monolitten pek farklı değildir. Sanal makineler yalıtım getirir ama ayağa kalkmaları uzun sürer ve kullanılmasalar da maliyet doğurur. Konteynerler işletim sistemi seviyesinde sanallaştırmadır, VM'lerin hafifletilmiş hâlidir; ama orkestrasyon (hangi anda hangi konteynerden kaç örnek çalışacak, işi bitenler nasıl kapatılacak) ciddi sorun olabilir. Serverless'te ortada ne VM ne konteyner vardır, yalnız servis vardır; servis istek geldiğinde Lambda fonksiyonu tarafından "uyandırılır", çalışır, sonucu döndürüp bekleme durumuna geçer.
- Bölüm 11: Üretime hazır servisler (ekrandan, 00:24:11)
  - Bir servisin üretime hazır olması için güvenlik, yapılandırılabilirlik ve gözlemlenebilirlik kalite özelliklerini karşılaması gerekir. Son başlık, bunları kolaylaştıran mikroservis gövdesi (microservice chassis) modelidir.
- Güvenlik
  - Uygulama geliştiricisinin sorumlu olduğu dört yön: kimlik doğrulama (authentication), yetkilendirme (authorization; genelde rol tabanlı güvenlik ve erişim kontrol listeleri), denetim (auditing; principal'ın işlemlerini izlemek) ve güvenli süreçler arası iletişim (ideal olarak bütün iletişim TLS üzerinden; servisler arası iletişimde bile kimlik doğrulama gerekebilir).
  - Monolitik FTGO'da güvenlik (ekrandan, 00:29:35 ve 00:30:40): istemci `POST /login` ile kullanıcı adı ve şifresini gönderir; Oturum Açma İşleyicisi (LoginHandler) kimliği doğrular, oturumu başlatır ve `Set-cookie: JSESSIONID=...` ile oturum jetonu çerezini döndürür. Sonraki isteklerde (`GET /orders/order-xyz`, `Cookie: JSESSIONID=...`) Oturum Tabanlı Güvenlik Önleyicisi (SessionBasedSecurityInterceptor) kullanıcı bilgisini oturumdan okur ve güvenlik bağlamını (security context) kurar; SiparişAyrıntılarıİstekİşleyicisi (OrderDetailsRequestHandler) gibi istek işleyicileri kullanıcı bilgisini bu bağlamdan alır. FTGO, Spring Security kullanan geleneksel bir Java EE uygulamasıdır; oturum bellek içi bir HTTP oturumudur, güvenlik bağlamı thread-local bir değişkende tutulur (`SecurityContextHolder.getContext().getAuthentication()`).
  - Mikroservislerde neden farklı: bellek içi güvenlik bağlamı kullanılamaz (servisler belleği paylaşmaz) ve merkezi bellek içi oturum anlamsızdır; veritabanı tabanlı ortak oturum ise gevşek bağlılık (loose coupling) ilkesini bozar.
  - API ağ geçidinde kimlik doğrulama (ekrandan, 00:33:10): API istemcileri kimlik bilgilerini her istekte `Authorization` başlığında gönderir; oturum tabanlı istemciler önce `POST /login` ile bir güvenlik jetonu alır, sonra her isteğe ekler. Ağ geçidi doğrular ve servise `SECURITY_TOKEN` ile iletir.
  - Yetkilendirme: kimlik doğrulama gerekli ama yetersizdir. Örnek: `getOrderDetails()`

sorgusunu yalnız siparişi veren tüketici ve ona yardım eden müşteri hizmetleri temsilcisi çağırabilir. Yetkilendirme API ağ geçidinde (yol bazında isteği reddetmek) ve servislerde (URL ve yöntem için rol tabanlı) uygulanabilir.
  - JWT (JSON Web Token): kullanıcı kimliği ve rolleri taşıyan şeffaf jeton; yükü (payload) kimlik, roller ve son kullanma tarihi gibi bilgileri içerir; yalnız oluşturan tarafın bildiği bir gizli anahtarla imzalanır, böylece taklit edilemez. Sorunu: kendi içinde bütün (self-contained) olduğu için geri alınamaz; kötü niyetli birinin eline geçen jetonu iptal etmenin pratik yolu yoktur. Çözüm kısa ömürlü jetonlardır; bu da jetonların sürekli yenilenmesini gerektirir.
  - OAuth 2.0 (ekrandan, 00:37:30 ve 00:38:46): API istemcisi için akış dört adımdır (Basic kimlik bilgisiyle istek → ağ geçidi `POST /oauth/token` ile şifre onayı (password grant) isteği → kimlik doğrulama sunucusu erişim ve yenileme jetonu döndürür → ağ geçidi `Authorization: Bearer` ile erişim jetonunu servislere iletir). Oturum tabanlı istemci için altı adımdır; jetonlar istemciye çerez olarak döner ve istemci sonraki isteklerde bunları gönderir.
- Yapılandırılabilirlik: dışsallaştırılmış yapılandırma (externalized configuration)
  - Yapılandırma çalışma zamanında servise dışarıdan verilir. İki model vardır.
  - İtme (push) modeli: dağıtım altyapısı yapılandırmayı ortam değişkenleri ya da yapılandırma dosyasıyla servis örneğine verir. En yaygın modeldir. Sınırları: çalışan servisi yeniden yapılandırmak zordur (çalışan sürecin ortam değişkenleri değiştirilemez) ve yapılandırma değerleri birçok servis tanımına dağılabilir.
  - Çekme (pull) modeli (ekrandan, 00:43:24): servis örneği açılışta yapılandırmayı bir yapılandırma sunucusundan okur (`getConfiguration("orderHistoryService")`); dağıtım altyapısı yalnız sunucuya erişim bilgisini (`CONFIG_SERVER_URL`) verir. Yapılandırma sunucusu Git, SQL/NoSQL veritabanı ya da Spring Cloud Config Server, HashiCorp Vault, AWS Parameter Store gibi özel sunucularla gerçeklenebilir.
  - Yapılandırma sunucusunun artıları: merkezi yapılandırma, hassas verilerin şifrelenmesi (bazı sunucular değeri servise vermeden önce çözer), yoklama (polling) ile dinamik yeniden yapılandırma. Eksisi: kurulup bakımı yapılması gereken ek bir altyapı parçasıdır.
- Gözlemlenebilirlik (ekrandan, 00:46:55)
  - Altı model: sağlık denetimi API'si, günlük (log) toplama, dağıtık izleme, istisna izleme, uygulama metrikleri, denetim günlüğü. Servisleri gözlemlenebilir yapmak geliştiricinin, toplanan bilgiyi işleyen altyapı operasyon ekibinin sorumluluğudur.
  - Sağlık denetimi API'si (ekrandan, 00:48:24): servis, dağıtım altyapısının (izleme sistemi, servis kayıt defteri vb.) periyodik çağırdığı bir uç nokta sunar; işleyici servisin veritabanı ve mesaj aracısı gibi dış bağlantılarını test eder. İki mesele: uç noktanın gerçeklenmesi ve altyapının bu uç noktayı çağıracak şekilde yapılandırılması.
  - Günlük toplama (ekrandan, 00:50:00): her servis örneğinin günlükleri bir günlük hattıyla merkezi günlük sunucusuna gider; arama ve uyarı oradan yapılır. Java günlük kütüphaneleri: Logback, Log4j, JUL. Yaygın altyapı ELK yığınıdır: Elasticsearch (metin arama odaklı NoSQL veritabanı, günlük sunucusu), Logstash (günlük hattı), Kibana (görselleştirme). Diğer hatlar: Fluentd, Apache Flume; bulut örneği: AWS CloudWatch Logs.
  - Dağıtık izleme (ekrandan, 00:53:16 ve 00:55:12): her dış isteğe benzersiz kimlik atanır, servisler arasında izlenir. İz (trace) bir dış isteği, yayılım (span) bir işlemi temsil eder (işlem adı, başlangıç ve bitiş zamanı); yayılımlar iç içe olabilir (üst/alt yayılım). İki parça: her serviste bir araç kütüphanesi (instrumentation library) ve dağıtık izleme sunucusu (ör. Open Zipkin). Kütüphane iz kimliğini giden isteklere başlık olarak ekler; araç mantığını iş mantığına karıştırmamak için interceptor ya da yön odaklı programlama (AOP) kullanılır.
  - Uygulama metrikleri: altyapı düzeyinden (CPU, bellek, disk) uygulama düzeyine (istek gecikmesi, istek sayısı, verilen/onaylanan/reddedilen sipariş sayısı) metrikler toplanır. Her metrik örneği: ad, değer, zaman damgası. Geliştirici servisini metrik toplayacak şekilde donatmak ve metrikleri metrik sunucusuna sunmakla sorumludur.
  - İstisna izleme (ekrandan, 00:59:22): servis istisnaları (ör. REST API ile `POST /exceptions`) bir istisna izleme hizmetine bildirir; hizmet istisnaları tekilleştirir, geliştiriciyi uyarır ve çözümünü izler. Örnek: Honeybadger (tamamen bulut tabanlı).
  - Denetim günlüğü (ekrandan, 01:00:00): her kullanıcının eylemlerini kaydeder (kullanıcı kimliği, eylem, iş nesnesi); genelde bir veritabanı tablosunda tutulur. Uygulama

yolları: iş mantığına kod eklemek, yön odaklı programlama, olay kaynaklama (event sourcing).
- Mikroservis gövdesi ve servis ağı (ekrandan, 01:03:34)
  - Mikroservis gövdesi (microservice chassis): dışsallaştırılmış yapılandırma, sağlık denetimi, metrikler, servis keşfi, devre kesici, dağıtık izleme gibi kaygıları üstlenen çerçeve ya da çerçeveler kümesidir; geliştirici çok az kod yazar ve iş mantığına odaklanır. FTGO'da Spring Boot ve Spring Cloud kullanılır.
  - Engeli: her programlama dili için ayrı bir gövde gerekir (Spring Boot/Cloud Node.js servisine yardım etmez).
  - Servis ağı (service mesh): bir servisin diğer servislerle ve dış uygulamalarla iletişimine aracılık eden ağ altyapısıdır. Devre kesici, dağıtık izleme, servis keşfi, yük dengeleme, kural tabanlı trafik yönlendirme ve TLS tabanlı IPC'yi servis dışında uygular. Gövdeye yalnız koda sıkı bağlı kaygılar (yapılandırma, sağlık denetimi) kalır.

Hocanın Özellikle Vurguladığı Kısımlar

- Serverless'teki süre sınırı bir tasarım uyarısıdır
  - Hoca, servisiniz Lambda süre sınırını aşacak kadar uzun çalışıyorsa büyük ihtimalle gerçekten "mikro" bir servis yazmadığınızı, yapıyı gözden geçirip gerçekten mikro servisler yazmak gerektiğini söyler.
- Dağıtım stratejilerinin evrimi: her adım bir öncekinin sorununu çözer
  - Tek makine → VM (yalıtım, ama yavaş açılış ve maliyet) → konteyner (hafif, ama orkestrasyon sorunu) → serverless (yalnız servis kalır, boşta kaynak harcanmaz). Serverless'in kazancı özellikle maliyettir: boş yere konteyner ya da sanal sunucu çalışmaz.
- Mikroserviste güvenlik monolitten taşınamaz
  - Hoca güvenlik kısmını şöyle toparlar: monolitteki authentication ve authorization mekanizmaları mikroservis ortamında aynı şekilde çalışmaz; farklı bir erişim jetonuna (access token) ihtiyaç vardır. Bu jeton, "gerçekten ben buyum ve bu hizmete erişmeye yetkiliyim" diyen bir rozet (badge) gibidir; OAuth 2.0 bunun nasıl alındığını tanımlar.
- Destek hizmetleri mikroservis ortamının ayrılmaz parçasıdır
  - Hoca sonda, mikroservis ortamının çalışması için güvenlikten kayıtlara, olağan dışı durumların ele alınmasına kadar monolitten farklı çözümler gerektiğini vurgular.

Kısa Tekrar Notları

- Üretim ortamı: servis yönetim arayüzü, çalışma zamanı servis yönetimi, izleme, istek yönlendirme.
- Host başına çok servis: verimli ve hızlı ama kaynak sınırı ve yalıtım yok.
- VM olarak servis: tam yalıtım ve bağımlılıklar imajda; büyük imaj, yavaş açılış, yüksek maliyet.
- Konteyner olarak servis: OS seviyesinde sanallaştırma, VM'den hafif ve hızlı; imaj ve altyapı yönetimi geliştiricide, orkestrasyon gerekir.
- Serverless (AWS Lambda): ZIP olarak yükle, istek başına öde; başlatma gecikmesi ve süre sınırı var.
- Üretime hazırlık: güvenlik, yapılandırılabilirlik, gözlemlenebilirlik.
- Güvenliğin dört yönü: kimlik doğrulama, yetkilendirme, denetim, güvenli IPC.
- Monolit: bellek içi oturum ve thread-local güvenlik bağlamı; mikroserviste ikisi de kullanılamaz.
- API ağ geçidi kimliği doğrular, jetonu servislere iletir; JWT şeffaf ve imzalıdır ama geri alınamaz, kısa ömürlü tutulur.
- OAuth 2.0: password grant ile erişim jetonu ve yenileme jetonu alınır, servislere Bearer jetonu gider.
- Yapılandırma: itme (ortam değişkeni, dosya) ve çekme (yapılandırma sunucusu) modelleri.
- Gözlemlenebilirlik: sağlık denetimi, günlük toplama (ELK), dağıtık izleme (iz ve yayılım, Zipkin), istisna izleme, metrikler, denetim günlüğü.
- Mikroservis gövdesi (Spring Boot/Cloud) dile bağlıdır; servis ağı bu işlevleri servis dışına

taşır.

Detaylı Açıklamalar

- Dağıtım stratejileri neden gerekir?
  - Bir uygulama, farklı dil ve çerçevelerle yazılmış onlarca servisten oluşabilir; her servis küçük bir uygulama olduğu için üretimde yüzlerce uygulama bulunabilir. Bu sayıda sunucuyu ve servisi elle yapılandırmak pratik değildir; mikroservisleri ölçekli dağıtmak için otomatik bir dağıtım süreci gerekir. Üretim ortamı şeklinde (ekrandan, 00:06:57) geliştirici izleme panolarıyla servisleri gözlemler, servis yönetim arayüzüyle servisleri yapılandırır; dağıtım hattı (deployment pipeline) servisleri bu arayüz üzerinden günceller; kullanıcı istekleri yönlendirme katmanından servislere ulaşır.
- Dört kalıbın karşılaştırması
  - Host başına çok servis, kaynak verimliliği ve dağıtım hızında öndedir; ama bir servisteki bellek sızıntısı bütün makineyi etkileyebilir ve yerleşim kararı zordur. VM olarak dağıtım bu sorunları yalıtım ve kaynak ayırma ile çözer, ama imaj büyüklüğü ve işletim sistemi açılışı süreyi uzatır, kullanılmayan kapasite için de ödeme yapılır. Konteyner, VM'nin yalıtım ve kaynak sınırlama artılarını korurken çok daha hafiftir. Serverless'te altyapı tamamen sağlayıcıdadır ve yalnız kullanılan süre ödenir; buna karşılık soğuk başlatma gecikmesi ve çalışma süresi sınırı vardır.
- Monolitten mikroservise güvenlik
  - Monolitte kullanıcı bir kez oturum açar, sunucu bellekte bir oturum tutar ve her istek bir çerezle bu oturuma bağlanır; istek işleyicileri kullanıcı bilgisini thread-local güvenlik bağlamından okur. Mikroserviste bir dış istek önce API ağ geçidinden geçer, sonra en az bir servise gider; servisler bellek paylaşmadığı için kimliğin başka bir mekanizmayla (jeton) taşınması gerekir. API ağ geçidi kimliği doğrular ve bir jeton üretir; bu jeton servislere iletilir. JWT bu iş için en yaygın şeffaf jetondur: servis, güvenlik servisine sormadan jetonun imzasını ve süresini doğrulayıp içindeki kimlik ve rolleri kullanabilir. Ancak aynı özellik jetonun iptalini imkânsızlaştırır; bu yüzden kısa ömürlü jetonlar ve OAuth 2.0'daki yenileme jetonu mekanizması kullanılır.
- OAuth 2.0 akışları (ekrandan, 00:37:30 ve 00:38:46)

```
API istemcisi:
1. İstemci -> API ağ geçidi:  GET /orders/1, Authorization: Basic ...
2. Ağ geçidi -> Spring OAuth2 kimlik doğrulama sunucusu:  POST /oauth/token (userid, passw
3. Sunucu -> ağ geçidi:  HTTP/1.1 200 OK, "access_token": ...
4. Ağ geçidi -> Sipariş servisi:  GET /orders/1, Authorization: Bearer <erişim jetonu>

Oturum tabanlı istemci:
1. İstemci -> oturum açma işleyicisi:  POST /login (userId, password)
2. Oturum açma işleyicisi -> kimlik doğrulama sunucusu:  POST /oauth/token
3. Sunucu -> işleyici:  erişim jetonu ve yenileme jetonu
4. Ağ geçidi -> istemci:  Set-Cookie: access_token=..., Set-Cookie: refresh_token=...
5. İstemci -> ağ geçidi:  GET /orders/1, Cookie: access_token=...; refresh_token=...
6. Oturum kimlik doğrulama önleyicisi jetonu doğrular -> Sipariş servisi:  Authorization:
```

- Dağıtık izleme örneği (ekrandan, 00:53:16 ve 00:55:12)
  - Zipkin ekranında bir `GET /orders/{orderId}` isteği iki yayılımdan oluşan bir iz olarak görünür: üst yayılım API ağ geçidindeki işlem (yaklaşık 18,9 ms), alt yayılım sipariş servisindeki işlem (yaklaşık 12,7 ms). Araç kütüphanesi, API ağ geçidinden sipariş servisine giden isteğe `X-B3-TraceId: XYZ` ve `X-B3-ParentSpanId: ABC` başlıklarını ekler; ağ geçidi yayılımını (iz XYZ, üst yayılım yok) ve sipariş servisi yayılımını (iz XYZ, üst yayılım ABC, yayılım DEF) dağıtık izleme sunucusuna bildirir; sunucu bunları birleştirip iz veritabanına yazar.
- Ders sonrası notlar

  - Sunumda AWS Lambda'nın en uzun çalışma süresi 300 saniye olarak verildi. Bu, kitabın yazıldığı dönemdeki sınırdı; büyük olasılıkla 2018 sonundan beri sınır 900 saniyedir (15 dakika).

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.